Université des Sciences et de la Technologie MB-Oran
Faculté des mathématiques et informatique
Département d'informatique
Niveau : Master 1

Polycopié de Cours — Bases de Données Avancées

CHAPITRE 3

Le modèle objet-relationnel

Chapitre 3 — Le modèle objet-relationnel

Ce chapitre présente les modèles qui étendent le relationnel avec les concepts de la programmation objet. On étudie d'abord le **modèle objet** et sa normalisation par l'ODMG (langages ODL, OQL et OML). On approfondit ensuite le **modèle relationnel-objet**, normalisé par SQL3 (SQL:1999), avec sa syntaxe de définition, de manipulation et d'interrogation. Les exemples utilisent la syntaxe d'Oracle, qui est le SGBD relationnel-objet le plus répandu dans l'enseignement.

Pourquoi aller au-delà du relationnel ?

Le modèle relationnel est simple, rigoureux et très performant pour les données de gestion. Il montre pourtant ses limites dès que les données deviennent complexes.

Limite du relationnel	Conséquence	Exemple
Types simples seulement (nombres, chaînes, dates)	Une donnée structurée doit être éclatée en plusieurs colonnes ou tables	Une adresse devient trois colonnes : rue, ville, code postal
Première forme normale (valeurs atomiques)	Un attribut multivalué impose une table séparée et une jointure	Les numéros de téléphone d'une personne
Pas de comportement associé aux données	Les traitements sont dispersés dans les applications	Le calcul de l'âge, répété dans chaque programme
Pas d'héritage	Les spécialisations sont simulées par plusieurs tables	Personne, Étudiant, Enseignant
Liens par valeur (clés étrangères)	La navigation entre objets passe toujours par des jointures	Un module et son responsable
Décalage avec les langages objet (*impedance mismatch*)	Il faut convertir les objets du programme en lignes, et inversement	Une application Java qui enregistre des objets

Ces limites sont particulièrement gênantes pour la conception assistée par ordinateur (CAO), le multimédia, les systèmes d'information géographique ou les données scientifiques. Deux réponses sont apparues : les **bases de données objet**, qui partent du monde objet, et les **bases de données relationnelles-objet**, qui étendent le relationnel.

3.1 Le modèle objet

3.1.1 Définition d'une base de données objet

Une **base de données objet** (BDO) stocke directement des **objets**, au sens de la programmation orientée objet : chaque donnée possède une identité, une structure, éventuellement complexe, et un comportement défini par des méthodes. Le **SGBD objet** (SGBDOO) qui la gère réunit deux familles de fonctions :

- celles d'un **langage objet** : objets complexes, identité, classes, encapsulation, héritage, polymorphisme ;
- celles d'un **SGBD** : persistance, gestion de grands volumes, partage concurrent, reprise après panne, langage de requêtes.

3.1.2 Les concepts fondamentaux

Concept	Définition	Exemple
Objet	Entité du monde réel, qui réunit un état (des valeurs) et un comportement (des méthodes)	L'étudiante Sara Haddad
Identité d'objet (OID)	Identifiant unique attribué par le système, indépendant des valeurs ; il ne change jamais	Deux étudiants homonymes restent deux objets distincts
Classe	Modèle commun à un ensemble d'objets : attributs et méthodes	La classe Etudiant
Attribut	Propriété de l'objet ; il peut être simple, structuré ou une collection	nom, adresse, téléphones

Concept	Définition	Exemple
Méthode	Opération attachée à la classe	age(), presenter()
Encapsulation	L'état n'est accessible qu'à travers les méthodes publiques	On appelle age() sans connaître le calcul
Objet complexe	Objet construit avec des constructeurs : tuple (struct), ensemble (set), liste (list), multi-ensemble (bag), tableau (array)	Une adresse (struct) et une liste de téléphones (list)
Association	Lien direct entre objets, par référence à leur OID	Un module référence son responsable
Héritage	Une sous-classe reprend les attributs et méthodes d'une super-classe et les complète	Etudiant hérite de Personne
Polymorphisme	Une même méthode a un comportement différent selon la classe de l'objet	presenter() diffère pour un étudiant et un enseignant
Extension (*extent*)	Ensemble des objets existants d'une classe ; c'est le point d'entrée des requêtes	L'ensemble de tous les étudiants

Identité ou valeur ? Dans le modèle relationnel, une ligne est identifiée par la valeur de sa clé primaire. Si cette valeur change, les clés étrangères qui y renvoient doivent changer aussi. Dans le modèle objet, l'OID est géré par le système et ne change jamais : deux objets aux valeurs identiques restent distincts, et les références entre objets restent valides.

3.1.3 Bilan des bases de données objet

Les SGBD objet (ObjectStore, Versant, O2, db4o, ObjectDB…) offrent une navigation très rapide entre objets et une intégration naturelle avec les langages objet. Leur diffusion est pourtant restée limitée : absence de fondement théorique comparable à l'algèbre relationnelle, langages de requêtes moins puissants que SQL, et coût de migration des bases relationnelles existantes. C'est ce qui a conduit à l'approche **relationnelle-objet** (section 3.3).

3.2 Le standard ODMG et ses langages

L'**ODMG** (*Object Data Management Group*) a proposé un standard pour les bases de données objet, dont la dernière version, ODMG 3.0, date de 2000. Il comprend un modèle objet commun et trois langages :

Langage	Rôle	Équivalent relationnel
ODL (*Object Definition Language*)	Définir le schéma : classes, attributs, associations, méthodes	LDD de SQL (CREATE TABLE)
OQL (*Object Query Language*)	Interroger les objets de façon déclarative	SELECT de SQL
OML (*Object Manipulation Language*)	Créer, modifier et supprimer des objets depuis un langage de programmation (C++, Java, Smalltalk)	INSERT, UPDATE, DELETE

Les exemples suivants portent sur une petite université : des personnes, qui sont des étudiants ou des enseignants, et des modules.

3.2.1 ODL : définir le schéma

ODL décrit chaque classe avec son **extension** (*extent*), sa **clé**, ses **attributs**, ses **associations** (*relationship*) et la signature de ses **méthodes**. Une association est déclarée des deux côtés, avec son **inverse** : le SGBD maintient ainsi automatiquement la cohérence des deux sens du lien.

```
class Personne (extent personnes key id) {
    attribute long   id;
    attribute string nom;
    attribute struct Adresse { string rue; string ville; } adresse;
    attribute list<string> telephones;
```

```
    short age();
};

class Etudiant extends Personne (extent etudiants) {
    attribute string matricule;
    relationship set<Module> suit inverse Module::inscrits;
};

class Enseignant extends Personne (extent enseignants) {
    attribute string grade;
    relationship set<Module> responsable_de inverse Module::responsable;
};

class Module (extent modules key code) {
    attribute string code;
    attribute string intitule;
    relationship Enseignant   responsable inverse Enseignant::responsable_de;
    relationship set<Etudiant> inscrits   inverse Etudiant::suit;
};
```

À retenir : **extends** exprime l'héritage, **struct** et **list** construisent des attributs complexes, et **relationship … inverse** remplace la clé étrangère et la table d'association du relationnel.

Le schéma suivant représente ces quatre classes. Il sert de repère pour lire les requêtes OQL de la section 3.2.2 : chaque flèche d'héritage correspond à un extends, chaque trait vert à une association déclarée avec son inverse, que l'on parcourt par une expression de chemin.

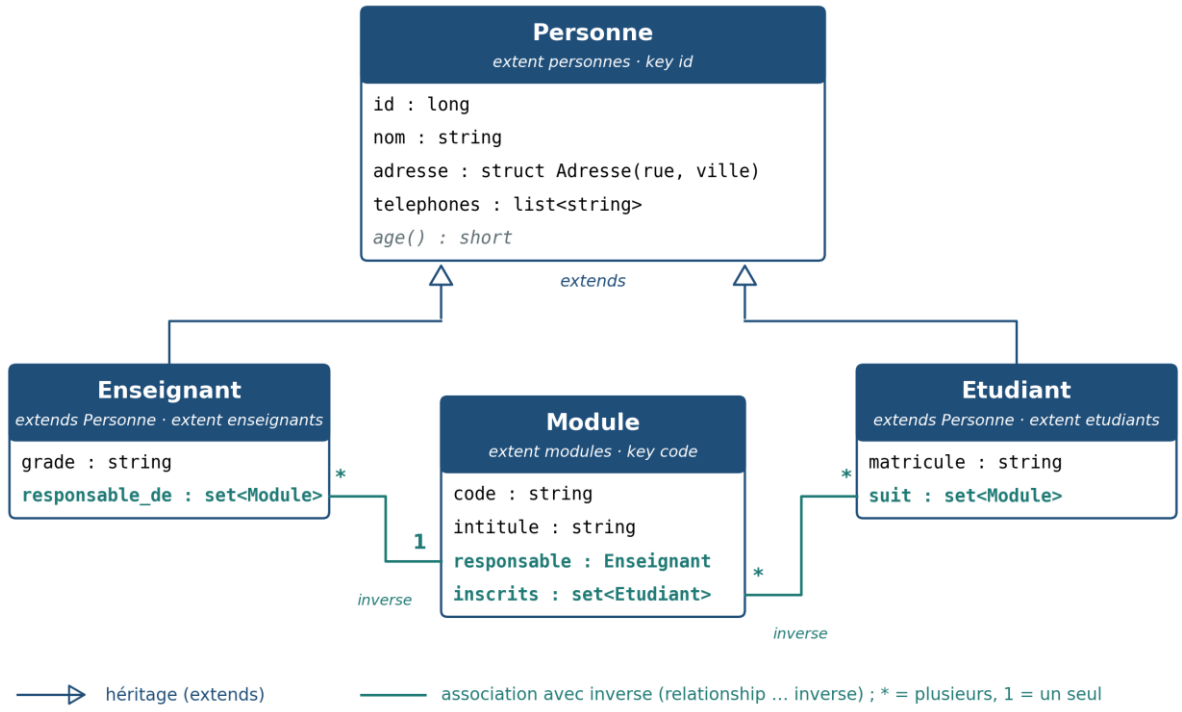


Schéma objet de l'université (ODL) : classes, héritage et associations

3.2.2 OQL : interroger les objets

OQL reprend la forme SELECT … FROM … WHERE de SQL. La clause FROM parcourt une **extension** ou une **collection**, et l'on navigue d'un objet à l'autre par des **expressions de chemin** (le point), sans jointure. Par exemple, m.responsable.nom suit l'association responsable du schéma ci-dessus, et e.suit parcourt l'ensemble des modules d'un étudiant.

```
-- Étudiants habitant Oran : chemin vers un attribut structuré
select e.nom
from e in etudiants
where e.adresse.ville = "Oran"

-- Chaque module avec le nom de son responsable : navigation par association
select m.intitule, m.responsable.nom
from m in modules

-- Étudiants inscrits au module BDA : parcours d'une collection
select e.nom
from e in etudiants, m in e.suit
where m.code = "BDA"

-- Appel de méthode dans une condition
select p.nom
from p in personnes
where p.age() > 30

-- Résultat structuré et agrégat sur une collection
select struct(nom: e.nom, nb_modules: count(e.suit))
from e in etudiants

-- Quantificateur existentiel
select e.nom
from e in etudiants
where exists m in e.suit : m.code = "BDA"
```

Une requête OQL peut renvoyer des valeurs, des objets ou des collections. Elle peut aussi s'imbriquer partout où une collection est attendue.

3.2.3 OML : manipuler les objets depuis un programme

L'ODMG ne définit pas un langage de manipulation autonome : l'OML est **lié à un langage hôte** (*language binding*). Les objets sont créés et modifiés comme des objets ordinaires du programme ; ils deviennent persistants lorsqu'ils sont rattachés à la base, à l'intérieur d'une transaction. Exemple illustratif avec la liaison Java de l'ODMG :

```
Database db = impl.newDatabase();
db.open("universite", Database.OPEN_READ_WRITE);

Transaction tx = impl.newTransaction();
tx.begin();
Etudiant e = new Etudiant("M1-2025-001", "Haddad");   // objet du programme
db.makePersistent(e);                                  // il devient persistant
e.setVille("Oran");                                    // simple appel de méthode
tx.commit();                                           // les changements sont
enregistrés

OQLQuery q = impl.newOQLQuery();
q.create("select e from e in etudiants where e.adresse.ville = $1");
q.bind("Oran");
Collection resultat = (Collection) q.execute();
```

Il n'y a ni INSERT ni conversion en lignes : le programme manipule directement les objets, et le SGBD se charge de leur persistance. C'est la réponse objet au décalage entre langages de programmation et bases de données.

3.3 Le modèle relationnel-objet

3.3.1 Principe

Le modèle **relationnel-objet** (RO) conserve le relationnel : tables, SQL, transactions, optimiseur, et lui ajoute les concepts objet les plus utiles. Plutôt que de remplacer les bases existantes, il les **étend** : une table peut contenir des colonnes de types complexes, des objets avec une identité, des références et des collections.

Cette approche a été normalisée par **SQL3**, c'est-à-dire **SQL:1999**, complété par SQL:2003. Elle est mise en œuvre, avec des variantes de syntaxe, par Oracle (depuis la version 8), IBM Db2 et PostgreSQL.

	Relationnel	**Objet (ODMG)**	**Relationnel-objet (SQL3)**
Structure de base	Table de lignes	Classe d'objets	Table de lignes ou d'objets
Types	Simples	Complexes	Simples et complexes
Identité	Valeur de la clé primaire	OID	OID pour les tables d'objets
Liens	Clés étrangères et jointures	Associations	Clés étrangères ou références (REF)
Comportement	Hors de la base	Méthodes	Méthodes des types
Héritage	Non	Oui	Oui, entre types
Langage	SQL	OQL et OML	SQL étendu (SQL3)

3.3.2 Les concepts du modèle RO

Concept	**Rôle**	**Syntaxe Oracle**
Type objet	Type défini par l'utilisateur, avec attributs et méthodes	CREATE TYPE … AS OBJECT
Corps du type	Code des méthodes	CREATE TYPE BODY
Méthode d'instance	Opération appliquée à un objet, qui y accède par SELF	MEMBER FUNCTION, MEMBER PROCEDURE
Constructeur	Fonction générée, du même nom que le type, qui crée un objet	adresse_t('rue', 'Oran', '31000')
Colonne objet	Colonne d'une table relationnelle dont le type est un type objet	adresse adresse_t
Table d'objets	Table dont chaque ligne est un objet doté d'un OID	CREATE TABLE … OF type
Référence	Pointeur vers l'OID d'un objet stocké dans une table d'objets	REF type, REF(), DEREF()
Tableau	Collection ordonnée, de taille maximale fixée	VARRAY(n) OF …
Table imbriquée	Collection non ordonnée et sans limite, stockée dans une table annexe	TABLE OF … et NESTED TABLE … STORE AS
Héritage	Un sous-type hérite des attributs et méthodes de son super-type	NOT FINAL et UNDER
Redéfinition	Un sous-type remplace une méthode héritée	OVERRIDING MEMBER FUNCTION
Substituabilité	Une table du super-type peut contenir des objets de ses sous-types	Table de personne_t contenant des etudiant_t

Colonne objet ou table d'objets ? Une colonne objet est une simple valeur structurée, sans identité propre : l'adresse d'une personne n'existe qu'à travers cette personne. Une table d'objets donne à chaque ligne un OID : ses objets peuvent alors être référencés depuis d'autres tables avec REF, comme un module référence son responsable.

VARRAY ou table imbriquée ? Le VARRAY convient à une petite liste ordonnée, de taille bornée et lue en bloc, comme les trois numéros de téléphone d'une personne. La table imbriquée convient à une collection plus

grande ou sans limite, dont on veut interroger, modifier ou supprimer des éléments un par un, comme les notes d'un étudiant.

3.4 Définition des données en SQL3

On construit pas à pas le schéma de l'université. Avec Oracle, chaque type se crée dans un ordre précis : un type doit exister avant d'être utilisé par un autre. Dans SQL*Plus, chaque CREATE TYPE et CREATE TYPE BODY se termine par une ligne contenant une barre oblique (/).

3.4.1 Types objets et méthodes

Un type objet simple, utilisé comme colonne structurée :

```
CREATE OR REPLACE TYPE adresse_t AS OBJECT (
  rue         VARCHAR2(60),
  ville       VARCHAR2(30),
  code_postal VARCHAR2(10)
);
/
```

Un tableau (VARRAY) d'au plus trois numéros de téléphone :

```
CREATE OR REPLACE TYPE telephones_t AS VARRAY(3) OF VARCHAR2(15);
/
```

Un type avec méthodes. La spécification déclare les attributs et la signature des méthodes. Le mot-clé NOT FINAL autorise la création de sous-types.

```
CREATE OR REPLACE TYPE personne_t AS OBJECT (
  id         NUMBER,
  nom        VARCHAR2(30),
  prenom     VARCHAR2(30),
  date_naiss DATE,
  adresse    adresse_t,
  telephones telephones_t,
  MEMBER FUNCTION age RETURN NUMBER,
  MEMBER FUNCTION presenter RETURN VARCHAR2
) NOT FINAL;
/
```

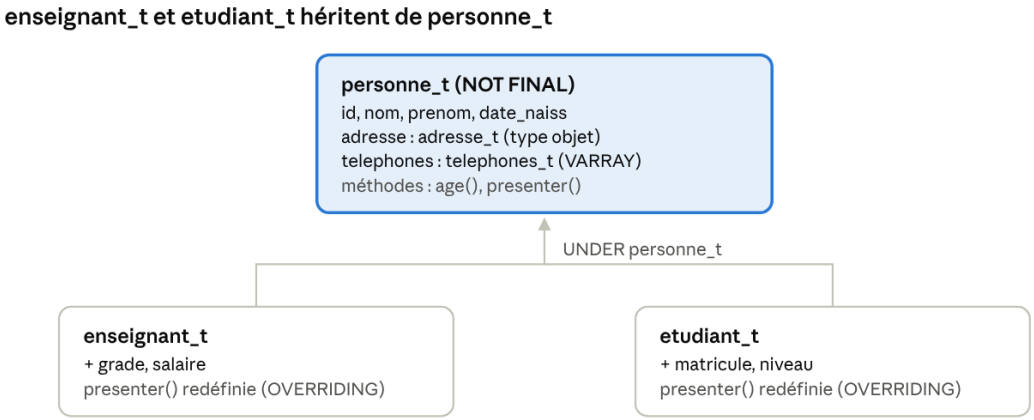
Le **corps du type** contient le code des méthodes. Une méthode accède aux attributs de l'objet directement, ou par SELF (SELF.nom).

```
CREATE OR REPLACE TYPE BODY personne_t AS
  MEMBER FUNCTION age RETURN NUMBER IS
  BEGIN
    RETURN TRUNC(MONTHS_BETWEEN(SYSDATE, date_naiss) / 12);
  END;

  MEMBER FUNCTION presenter RETURN VARCHAR2 IS
  BEGIN
    RETURN prenom || ' ' || nom;
  END;
END;
/
```

3.4.2 Héritage et redéfinition

Un sous-type se déclare avec UNDER. Il hérite de tous les attributs et méthodes de personne_t, en ajoute de nouveaux et peut **redéfinir** une méthode avec OVERRIDING.



hiérarchie des types personne_t · un super-type, deux sous-types

```
CREATE OR REPLACE TYPE enseignant_t UNDER personne_t (
  grade   VARCHAR2(20),
  salaire NUMBER,
  OVERRIDING MEMBER FUNCTION presenter RETURN VARCHAR2
);
/
CREATE OR REPLACE TYPE BODY enseignant_t AS
  OVERRIDING MEMBER FUNCTION presenter RETURN VARCHAR2 IS
  BEGIN
    RETURN grade || ' ' || prenom || ' ' || nom;
  END;
END;
/

CREATE OR REPLACE TYPE etudiant_t UNDER personne_t (
  matricule VARCHAR2(12),
  niveau    VARCHAR2(5),
  OVERRIDING MEMBER FUNCTION presenter RETURN VARCHAR2
);
/
CREATE OR REPLACE TYPE BODY etudiant_t AS
  OVERRIDING MEMBER FUNCTION presenter RETURN VARCHAR2 IS
  BEGIN
    RETURN 'Étudiant ' || niveau || ' : ' || prenom || ' ' || nom;
  END;
END;
/
```

Si l'on ne voulait jamais créer de « simple » personne, on déclarerait personne_t avec NOT INSTANTIABLE NOT FINAL : seuls ses sous-types pourraient alors avoir des instances.

3.4.3 Tables d'objets et références

Une **table d'objets** se crée avec OF. Chaque ligne est un objet doté d'un OID. Les contraintes se déclarent sur les attributs du type.

```
CREATE TABLE personnes OF personne_t (
  id  PRIMARY KEY,
  nom NOT NULL
);
```

Cette table est **substituable** : elle peut contenir des objets personne_t, mais aussi des enseignant_t et des etudiant_t.

Un module **référence** son responsable par son OID, avec un attribut de type REF. La clause SCOPE IS limite les références aux objets de la table personnes.

```
CREATE OR REPLACE TYPE module_t AS OBJECT (
  code        VARCHAR2(10),
  intitule    VARCHAR2(60),
  coef        NUMBER,
  responsable REF personne_t
);
/

CREATE TABLE modules OF module_t (
  code        PRIMARY KEY,
  responsable SCOPE IS personnes
);
```

3.4.4 Collections : la table imbriquée

Les notes d'un étudiant forment une collection d'objets note_t. On la déclare comme une **table imbriquée** (TABLE OF), puis on l'utilise comme colonne d'une table ordinaire. La clause NESTED TABLE … STORE AS désigne la table annexe où Oracle range physiquement les éléments.

```
CREATE OR REPLACE TYPE note_t AS OBJECT (
  code_module VARCHAR2(10),
  note        NUMBER(4,2)
);
/
CREATE OR REPLACE TYPE notes_t AS TABLE OF note_t;
/

CREATE TABLE releves (
  matricule VARCHAR2(12) PRIMARY KEY,
  annee     NUMBER(4),
  notes     notes_t
) NESTED TABLE notes STORE AS releves_notes;
```

La table releves est une table relationnelle classique : le modèle RO permet de mélanger colonnes simples et colonnes complexes.

3.4.5 Faire évoluer et supprimer les types

Opération	Syntaxe
Ajouter un attribut et propager le changement aux tables	ALTER TYPE personne_t ADD ATTRIBUTE email VARCHAR2(60) CASCADE;
Supprimer un type qui n'est plus utilisé	DROP TYPE note_t;
Supprimer un type malgré ses dépendances	DROP TYPE note_t FORCE;

Un type utilisé par une table ou par un autre type ne peut pas être supprimé directement : il faut d'abord supprimer les tables et les types qui en dépendent, dans l'ordre inverse de leur création.

3.4.6 Vue d'ensemble du schéma

Le schéma ci-dessous rassemble les types et les trois tables créés dans cette section. Il sert de repère pour les manipulations et les requêtes des sections 3.5 et 3.6 :

personnes est une table d'objets substituable : elle contient des enseignants et des étudiants, chacun doté d'un OID.

modules référence son responsable par une REF vers l'OID d'un objet de personnes : on y navigue sans jointure (m.responsable.nom).

releves est une table relationnelle dont la colonne notes est une table imbriquée, stockée physiquement dans releves_notes. Elle n'est reliée aux étudiants que par la valeur du matricule : la requête doit alors utiliser une jointure avec TREAT.

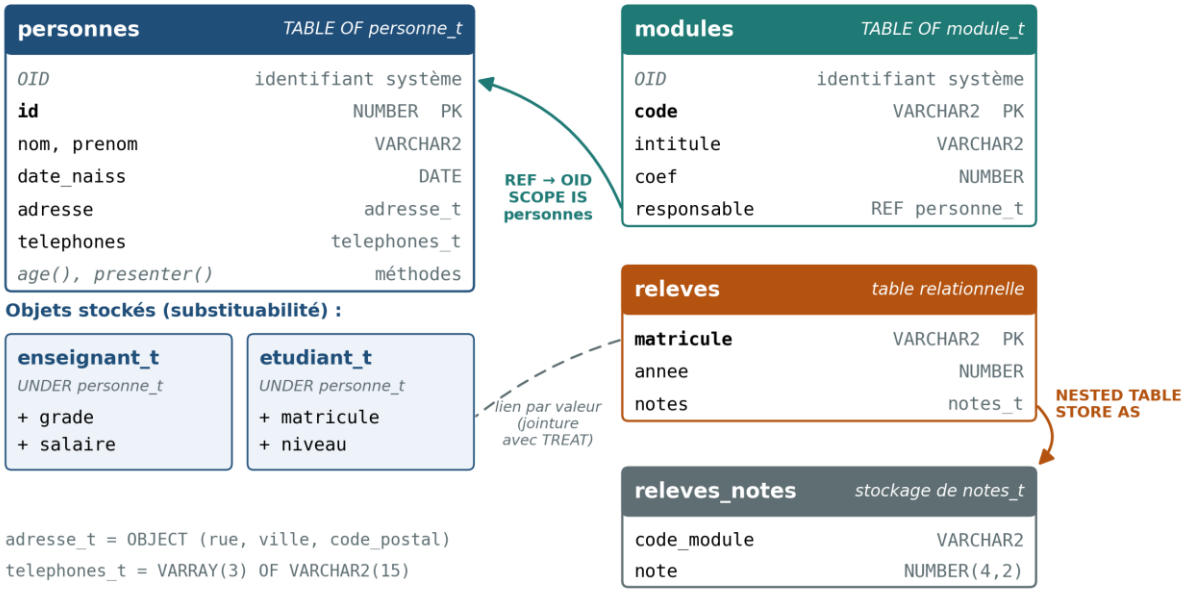


Schéma de la base universitaire : types, tables d'objets, référence et table imbriquée

3.5 Manipulation des données en SQL3

3.5.1 Insérer des objets

On crée un objet en appelant le **constructeur** de son type, qui porte le même nom que lui et prend les attributs dans l'ordre de leur déclaration. Les constructeurs s'imbriquent pour les attributs complexes.

Dans une table d'objets substituable. On peut y insérer des objets de chaque sous-type :

```
INSERT INTO personnes VALUES (
  enseignant_t(1, 'Benali', 'Karim', DATE '1975-03-12',
               adresse_t('Rue Khemisti', 'Oran', '31000'),
               telephones_t('0550123456', '041456789'),
               'Professeur', 180000));

INSERT INTO personnes VALUES (
  enseignant_t(2, 'Mansouri', 'Leila', DATE '1982-11-05',
               adresse_t('Rue Didouche Mourad', 'Alger', '16000'),
               telephones_t('0661234567'),
               'MCA', 140000));

INSERT INTO personnes VALUES (
  etudiant_t(10, 'Haddad', 'Sara', DATE '2002-06-01',
             adresse_t('Cité USTO', 'Oran', '31000'),
             telephones_t('0770123456'),
             'M1-2025-001', 'M1'));

INSERT INTO personnes VALUES (
  etudiant_t(11, 'Kaci', 'Amine', DATE '2001-09-20',
             adresse_t('Boulevard de la Soummam', 'Oran', '31000'),
             telephones_t('0555987654'),
             'M1-2025-002', 'M1'));
```

Avec une référence. La fonction REF(p) renvoie la référence de l'objet p. On l'obtient par un SELECT sur la table d'objets :

```
INSERT INTO modules
SELECT 'BDA', 'Bases de données avancées', 3, REF(p)
```

```
FROM personnes p WHERE p.id = 1;

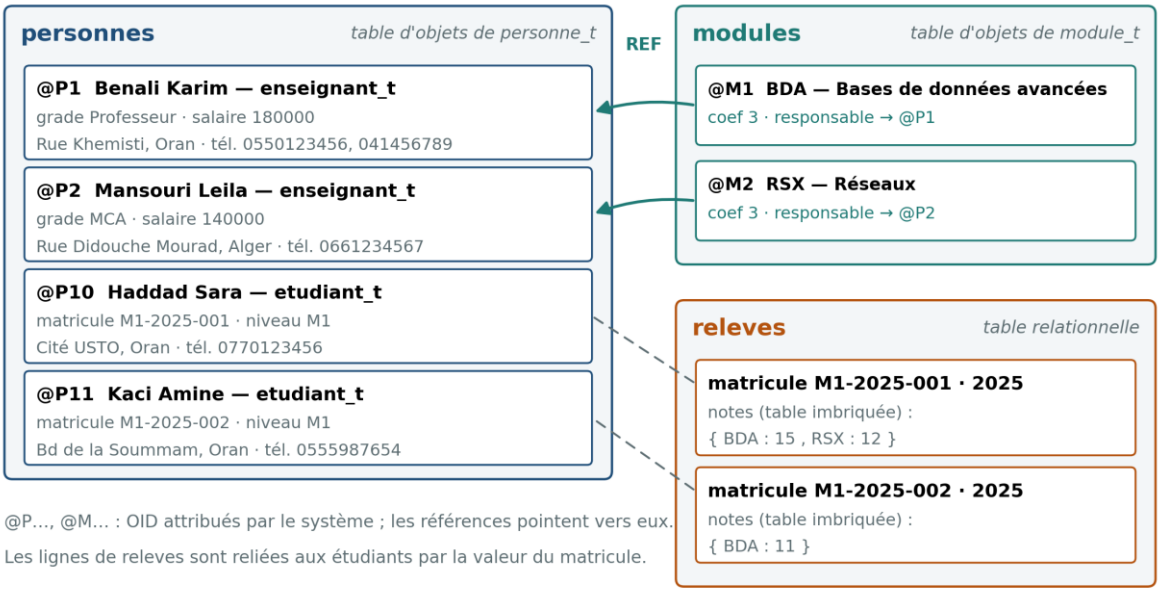
INSERT INTO modules
SELECT 'RSX', 'Réseaux', 3, REF(p)
FROM personnes p WHERE p.id = 2;
```

Avec une collection. Le constructeur de la collection reçoit la liste de ses éléments :

```
INSERT INTO releves VALUES ('M1-2025-001', 2025,
  notes_t(note_t('BDA', 15), note_t('RSX', 12)));

INSERT INTO releves VALUES ('M1-2025-002', 2025,
  notes_t(note_t('BDA', 11)));
```

Contenu de la base après les insertions. Le schéma suivant montre l'état des trois tables une fois toutes les instructions INSERT exécutées. Les OID (@P1, @M1…) sont attribués par le système : chaque module pointe par une référence vers son responsable, et chaque relevé n'est relié à son étudiant que par la valeur du matricule. C'est sur cet état que portent les requêtes de la section 3.6.



Contenu des tables personnes, modules et releves après les insertions de la section 3.5.1

3.5.2 Modifier et supprimer

Les exemples ci-dessous montrent la syntaxe. La section 3.6 repart de l'état obtenu en 3.5.1, sans ces modifications.

Modifier un attribut d'une colonne objet. On suit le chemin depuis un alias de table, qui est obligatoire :

```
UPDATE personnes p
SET p.adresse.ville = 'Mostaganem'
WHERE p.id = 11;
```

Remplacer un VARRAY. Un tableau se modifie en bloc, en le remplaçant entièrement :

```
UPDATE personnes p
SET p.telephones = telephones_t('0770112233', '041998877')
WHERE p.id = 10;
```

Changer une référence. On affecte la référence d'un autre objet :

```
UPDATE modules m
SET m.responsable = (SELECT REF(p) FROM personnes p WHERE p.id = 1)
WHERE m.code = 'RSX';
```

Modifier les éléments d'une table imbriquée. L'opérateur TABLE( ) transforme la collection d'une ligne en une table que l'on peut traiter avec INSERT, UPDATE ou DELETE :

```
-- Ajouter une note
INSERT INTO TABLE (SELECT r.notes FROM releves r WHERE r.matricule = 'M1-2025-001')
VALUES ('SD', 14);

-- Modifier une note
UPDATE TABLE (SELECT r.notes FROM releves r WHERE r.matricule = 'M1-2025-001') n
SET n.note = 16
WHERE n.code_module = 'BDA';

-- Supprimer une note
DELETE FROM TABLE (SELECT r.notes FROM releves r WHERE r.matricule = 'M1-2025-001') n
WHERE n.code_module = 'SD';
```

Supprimer un objet. La suppression s'écrit comme en relationnel :

```
DELETE FROM personnes p WHERE p.id = 2;
```

Attention : une référence n'est pas une clé étrangère. Après cette suppression, le module RSX garde une référence vers un objet qui n'existe plus : on dit que la référence est **pendante** (*dangling*). La section 3.6 montre comment la détecter.

3.6 Interrogation en SQL3

Les résultats sont donnés sur les données insérées en 3.5.1. Pour suivre les chemins et les références utilisés, reportez-vous au schéma de la section 3.4.6. Le contenu de chaque table est représenté à la fin de la section 3.5.1.

3.6.1 Chemins et méthodes

Accéder à un attribut structuré par une expression de chemin, toujours à partir d'un alias :

```
SELECT p.nom, p.adresse.ville AS ville
FROM personnes p;
```

nom	ville
Benali	Oran
Mansouri	Alger
Haddad	Oran
Kaci	Oran

Appeler une méthode, avec des parenthèses même sans paramètre :

```
SELECT p.nom, p.age() AS age
FROM personnes p
WHERE p.age() > 30;
```

Résultat : **Benali (51 ans)** et **Mansouri (43 ans)**.

Polymorphisme. La même méthode appelle la version propre au type réel de chaque objet :

```
SELECT p.presenter() AS presentation
FROM personnes p;
```

presentation
Professeur Karim Benali
MCA Leila Mansouri
Étudiant M1 : Sara Haddad
Étudiant M1 : Amine Kaci

3.6.2 Objets et sous-types

Récupérer l'objet entier avec VALUE( ) :

```
SELECT VALUE(p) FROM personnes p WHERE p.id = 1;
```

Le résultat est un objet complet : ENSEIGNANT_T(1, 'Benali', 'Karim', …, 'Professeur', 180000).

Filtrer par type avec IS OF. Le mot-clé ONLY exclut les sous-types :

```
SELECT p.nom FROM personnes p
WHERE VALUE(p) IS OF (etudiant_t);          -- Haddad, Kaci

SELECT p.nom FROM personnes p
WHERE VALUE(p) IS OF (ONLY personne_t);     -- aucune ligne
```

Lire un attribut d'un sous-type avec TREAT, qui « convertit » l'objet vers le sous-type. Pour un objet d'un autre type, TREAT renvoie NULL.

```
SELECT p.nom, TREAT(VALUE(p) AS enseignant_t).grade AS grade
FROM personnes p
WHERE VALUE(p) IS OF (enseignant_t);
```

nom	grade
Benali	Professeur
Mansouri	MCA

3.6.3 Références

Naviguer par une référence, sans jointure : Oracle déréférence automatiquement m.responsable.

```
SELECT m.code, m.responsable.nom AS responsable
FROM modules m;
```

code	responsable
BDA	Benali
RSX	Mansouri

Obtenir l'objet référencé avec DEREF( ), et **comparer des références** avec REF( ) :

```
SELECT DEREF(m.responsable) FROM modules m WHERE m.code = 'BDA';

SELECT m.code
FROM modules m
WHERE m.responsable = (SELECT REF(p) FROM personnes p WHERE p.nom = 'Benali');
```

La seconde requête renvoie **BDA**. Elle équivaut à la condition WHERE m.responsable.nom = 'Benali'.

Détecter les références pendantes. Après la suppression de Mansouri (section 3.5.2) :

```
SELECT m.code FROM modules m WHERE m.responsable IS DANGLING;
```

Résultat : **RSX**, dont le responsable n'existe plus.

3.6.4 Collections

Déplier une table imbriquée avec TABLE( ) dans la clause FROM : on obtient une ligne par élément, comme après une jointure.

```
SELECT r.matricule, n.code_module, n.note
FROM releves r, TABLE(r.notes) n;
```

matricule	code_module	note
M1-2025-001	BDA	15

matricule	code_module	note
M1-2025-001	RSX	12
M1-2025-002	BDA	11

Calculer sur une collection, avec une sous-requête sur TABLE( ) :

```
SELECT r.matricule,
       (SELECT AVG(n.note) FROM TABLE(r.notes) n) AS moyenne
FROM releves r;
```

Résultat : **13,5** pour M1-2025-001 ((15 + 12) / 2) et **11** pour M1-2025-002.

Relier une collection à une table d'objets. On retrouve le nom de l'étudiant à partir de son matricule, attribut du sous-type etudiant_t :

```
SELECT p.nom, n.code_module, n.note
FROM releves r, TABLE(r.notes) n, personnes p
WHERE TREAT(VALUE(p) AS etudiant_t).matricule = r.matricule;
```

Résultat : Haddad → BDA 15 et RSX 12 ; Kaci → BDA 11.

Déplier un VARRAY. Ses éléments, de type simple, se lisent dans la pseudo-colonne COLUMN_VALUE :

```
SELECT p.nom, t.COLUMN_VALUE AS telephone
FROM personnes p, TABLE(p.telephones) t;
```

Benali apparaît deux fois, une fois par numéro ; chacune des trois autres personnes apparaît une fois.

3.6.5 Récapitulatif des opérateurs

Opérateur	Rôle
p.adresse.ville	Accéder à un attribut par un chemin
p.age()	Appeler une méthode
VALUE(p)	Obtenir l'objet d'une table d'objets
REF(p)	Obtenir la référence (l'OID) d'un objet
DEREF(r)	Obtenir l'objet pointé par une référence
IS OF (type), IS OF (ONLY type)	Tester le type d'un objet
TREAT(objet AS sous_type)	Voir un objet comme un sous-type
IS DANGLING	Tester si une référence pointe vers un objet supprimé
TABLE(collection)	Traiter une collection comme une table
COLUMN_VALUE	Élément d'une collection de type simple

3.7 Synthèse

- Le **modèle objet** stocke des objets dotés d'une identité (OID), d'une structure complexe et de méthodes. L'**ODMG** l'a normalisé avec trois langages : **ODL** pour définir le schéma, **OQL** pour l'interroger, **OML** pour manipuler les objets depuis un langage de programmation.
- Le **modèle relationnel-objet** étend le relationnel au lieu de le remplacer. Normalisé par **SQL3 (SQL:1999)**, il ajoute à SQL les types objets et leurs méthodes, l'héritage, les tables d'objets, les références (REF) et les collections (VARRAY, tables imbriquées).
- En **définition**, on crée les types (CREATE TYPE, CREATE TYPE BODY, UNDER), puis les tables (CREATE TABLE … OF, NESTED TABLE … STORE AS).
- En **manipulation**, on utilise les constructeurs, REF( ) pour les références et TABLE( ) pour modifier les collections.
- En **interrogation**, on navigue par des chemins et des références, on appelle des méthodes, on teste les sous-types avec IS OF et TREAT, et l'on déplie les collections avec TABLE( ).

Les autres SGBD. La syntaxe présentée est celle d'Oracle. PostgreSQL propose des notions voisines avec une autre syntaxe : types composites (CREATE TYPE … AS), tableaux (type[]) et héritage entre tables (INHERITS).